

**Raport de interpretare a rezultatelor analizei
învățării automate**

Materia : Inteligență Artificială

Munteanu Irina Daniela 332CC

1. Introducere

În contextul dezvoltării sistemelor moderne bazate pe inteligență artificială, metodele de învățare automată (Machine Learning) joacă un rol esențial în extragerea de informații relevante din date și în realizarea de predicții automate. Această lucrare are ca scop familiarizarea cu etapele fundamentale ale unui pipeline de Machine Learning, pornind de la explorarea datelor și preprocesare, până la antrenarea și evaluarea unor modele predictive.

Problema abordată în cadrul acestei teme constă în utilizarea unui set de date de tip tabelar pentru rezolvarea a două sarcini distincte:

- o problemă de **clasificare multclasă**, având ca obiectiv prezicerea variabilei *vacation*, care descrie tipul de vacanță accesibil unui angajat;
- o problemă de **regresie**, având ca obiectiv estimarea variabilei *salary*, reprezentând venitul asociat unui profil profesional.

Setul de date utilizat conține informații despre angajați, incluzând atât attribute numerice (de exemplu, ani de experiență sau număr de certificări), cât și attribute categoriale (nivelul educațional sau industria).

Obiectivul principal al lucrării este dezvoltarea unor modele predictive eficiente, precum și analiza impactului diferitelor etape de preprocesare și al alegerii parametrilor asupra performanței acestora.

2. Încărcarea și pregătirea inițială a datelor

În prima etapă a rezolvării, setul de date a fost încărcat utilizând biblioteca *pandas*, o unealtă standard pentru manipularea datelor tabelare în Python. Fișierul de antrenare a fost citit într-o structură de tip *DataFrame*, facilitând accesul și prelucrarea ulterioară a atributelor.

```
df_train = pd.read_csv('CC_education_economy_train.csv')
```

3. Explorarea datelor (Exploratory Data Analysis)

3.1 Prezentarea generală

În prima etapă a analizei exploratorii, s-a realizat o inspecție generală a setului de date, utilizând funcții standard pentru vizualizarea structurii acestuia.

Această analiză preliminară permite identificarea tipurilor de attribute (numerice sau categoriale), precum și observarea distribuțiilor inițiale ale datelor.

3.2 Analiza valorilor lipsă

Un aspect esențial al analizei exploratorii îl constituie identificarea valorilor lipsă. Acestea pot afecta semnificativ performanța modelelor de învățare automată.

job_title	0
experience_years	0
education_level	0
skills_count	0
industry	0
company_size	0
location	0
remote_work	19153
certifications	0
salary	0
total_days_worked	0
aggregated_score	0
skill_bracket	0
vacation	0
dtype: int64	

	count_missing	percentage
remote_work	19153	29.93

Analiză: Din afișarea din python a tabelului, cât și a desenului aferent, lipsesc date doar din variabila `remote_work` în proporție de aproximativ ~30%, ce reprezintă dacă angajatul lucrează online, din totalul de 64000 de date de train doar $64000 - 19153 = 44847$ date având câmpurile complete.

3.3 Analiza tipurilor de attribute și a plajei de valori ale acestora

```
df_train.head()
df_train.info()
```

	job_title	experience_years	education_level	skills_count	industry	company_size	location	remote_work	certifications	salary	total_days_worked	aggregated_score	skill_bracket	vacation
0	Backend Developer	19	High School	12	Media	Large	USA	Yes	0	203997	4560	-0.72	mid	Small
1	Backend Developer	4	High School	13	Telecom	Small	Germany	NaN	5	116239	960	-0.48	high	No Vacation
2	Backend Developer	8	High School	16	Retail	Enterprise	Remote	No	4	145802	1920	-1.53	high	No Vacation
3	Backend Developer	12	High School	9	Telecom	Enterprise	USA	Hybrid	2	190299	2880	0.53	mid	Small
4	Backend Developer	14	High School	6	Telecom	Large	UK	Hybrid	4	152679	3360	-0.19	low	No Vacation

```

#   Column      Non-Null Count  Dtype
---  -
0   job_title    64000 non-null    str
1   experience_years  64000 non-null    int64
2   education_level  64000 non-null    str
3   skills_count    64000 non-null    int64
4   industry       64000 non-null    str
5   company_size    64000 non-null    str
6   location        64000 non-null    str
7   remote_work     44847 non-null    str
8   certifications   64000 non-null    int64
9   salary          64000 non-null    int64
10  total_days_worked 64000 non-null    int64
11  aggregated_score  64000 non-null    float64
12  skill_bracket    64000 non-null    str
13  vacation         64000 non-null    str
dtypes: float64(1), int64(5), str(8)
memory usage: 6.8 MB

```

Din afișarea anterior pusă în legătură cu detalii despre tipul de structura al atributelor, am concluzionat că există:

- 6 attribute numerice : *experience_years*, *skills_count*, *certifications*, *salary*, *total_days_worked*, *aggregated_score*
- 8 attribute categoriale, dintre care 4 ordinale : *job_title*, *industry*, *location*, *remote_work*, *education_level* (High School < Diploma < Bachelor < Master < PhD) , *company_size* (Small < Medium < Large < Enterprise) , *vacation* (No Vacation < Small < Medium < Large), *skill_bracket* (low < mid < high)

Pentru attribute numerice continue , cu ajutorul funcției `describe()`, am extras următorul tabel:

	experience_years	skills_count	certifications	total_days_worked	aggregated_score	salary
count	64000.00	64000.00	64000.00	64000.00	64000.00	64000.00
mean	9.97	11.60	2.51	2393.51	0.00	145683.57
std	6.05	10.74	1.71	1452.04	1.00	37384.76
min	0.00	1.00	0.00	-1.00	-4.80	31867.00
25%	5.00	5.00	1.00	1200.00	-0.67	119209.75
50%	10.00	10.00	3.00	2400.00	0.00	143299.00
75%	15.00	15.00	4.00	3600.00	0.68	169582.25
max	20.00	69.00	5.00	4800.00	4.22	333046.00

Analiză:

Pentru atributul *skills_count*, s-a observat că valoarea maximă este semnificativ mai mare decât percentila 75% , existând valori care ajung până la 69. Această discrepanță indică prezența unor outlieri. Astfel de valori extreme pot afecta negativ modelele, în special pe cele bazate pe regresie liniară, deoarece acestea sunt sensibile la valori mari și pot ajusta

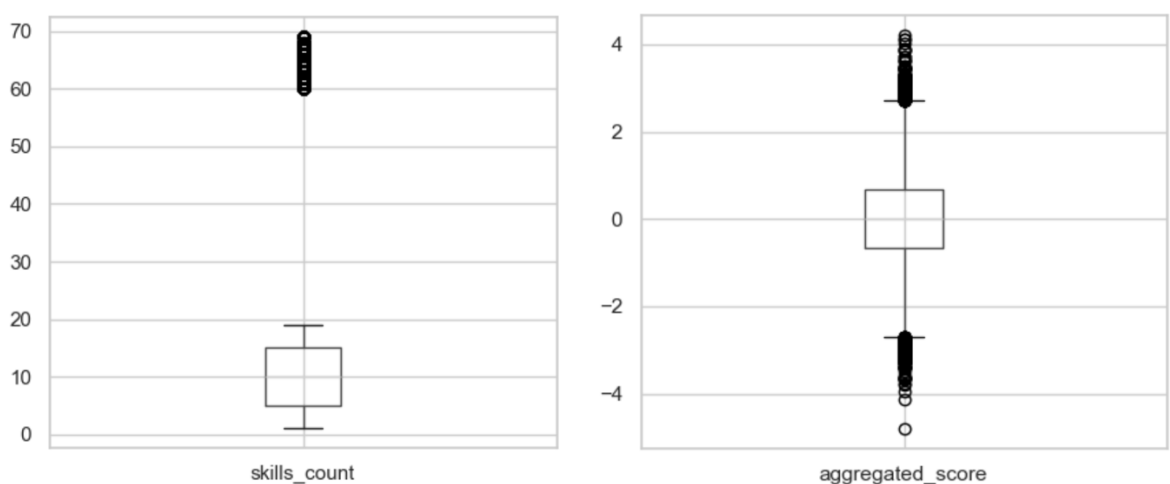
coeficienții în mod disproporționat pentru a acomoda aceste cazuri izolate. În consecință, modelul poate deveni mai puțin precis pentru majoritatea datelor.

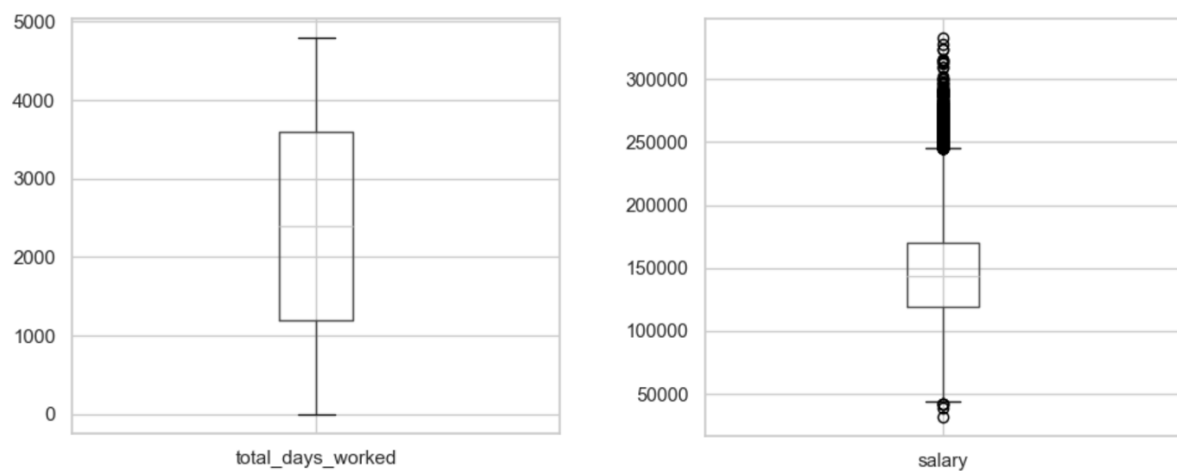
În cazul atributului *total_days_worked*, am identificat valoarea minimă ca fiind negativă (-1) care nu are semnificație reală . Aceste valori reprezintă, cel mai probabil, erori sau valori lipsă codificate incorect.

Pentru atributul *salary*, distribuția este ușor asimetrică spre dreapta, însă fără prezența unor outlieri extremi. Această asimetrie poate influența performanța modelelor liniare, deoarece acestea presupun, în mod ideal, o distribuție aproximativ normală a datelor. Totuși, impactul este mai redus comparativ cu cazul outlierilor severi.

Pentru a reduce influența acestor probleme asupra performanței modelelor, putem pentru :

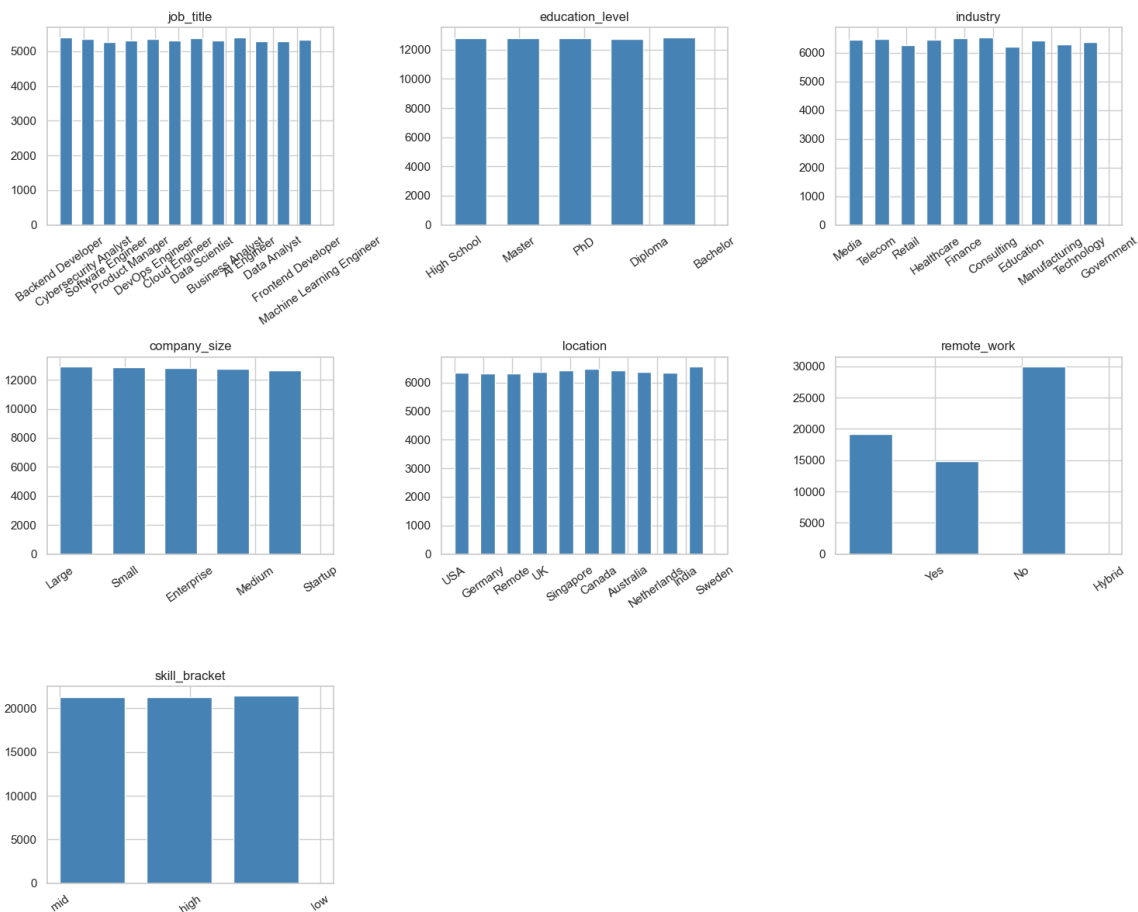
- *skills_count*, valorile extreme pot fi tratate fie prin eliminare, fie prin înlocuire (imputare), astfel încât să se limiteze impactul lor asupra modelului
- *total_days_worked*, valorile invalide (negative) vor fi tratate ca valori lipsă și imputate utilizând o metodă adecvată (de exemplu, media sau mediana)
- *salary*, se poate aplica o transformare logaritmică sau o standardizare, pentru a reduce efectele asimetriei și a stabiliza variația datelor.





Pentru attribute discrete sau ordinale, cu ajutorul bibliotecii DataFrame, a rezultat un tabel în care să imi afişez attributele categorice cu : numărul de valori lipsă, valori unice, cât şi toate valorile prezentate pentru fiecare:

	fara_valori_lipsa	valori_unice	valori
job_title	64000	12	[Backend Developer, Cybersecurity Analyst, Sof...
education_level	64000	5	[High School, Master, PhD, Diploma, Bachelor]
industry	64000	10	[Media, Telecom, Retail, Healthcare, Finance, ...
company_size	64000	5	[Large, Small, Enterprise, Medium, Startup]
location	64000	10	[USA, Germany, Remote, UK, Singapore, Canada, ...
remote_work	44847	3	[Yes, No, Hybrid]
skill_bracket	64000	3	[mid, high, low]



Analiză:

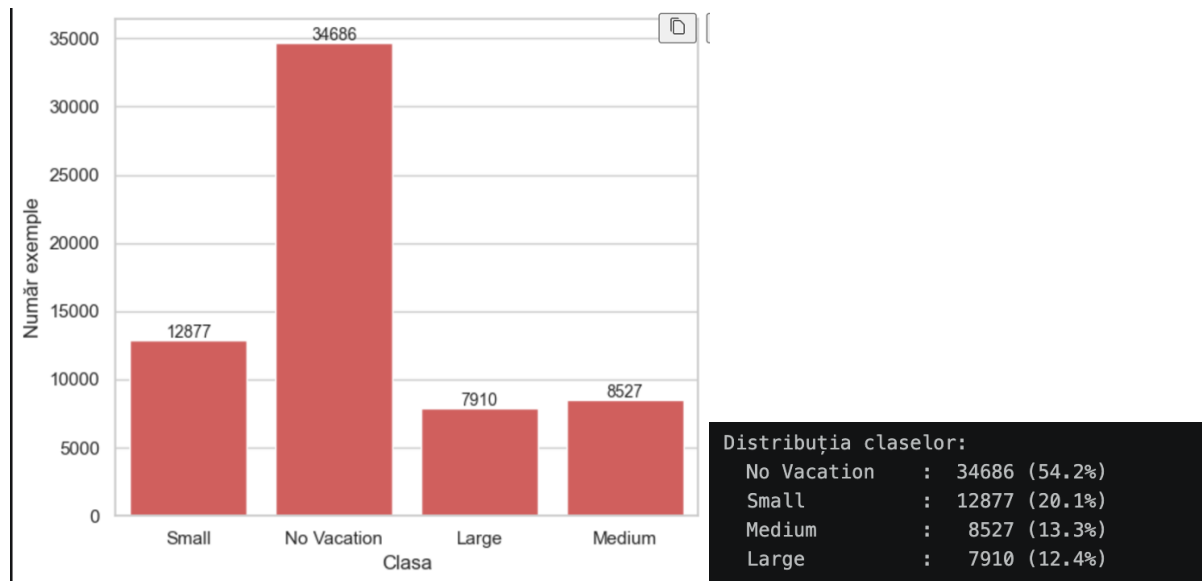
Pentru atributul *remote_work*, histograma evidențiază prezența unui procent semnificativ de valori lipsă (aproximativ 30%). Această proporție ridicată poate afecta negativ modelele de învățare automată, deoarece mulți algoritmi nu pot gestiona direct valori lipsă. În plus, ignorarea acestor valori sau eliminarea rândurilor corespunzătoare ar putea conduce la pierderea unei cantități importante de informație, afectând capacitatea de generalizare a modelului.

Pentru atributul *skill_bracket*, observăm că are doar trei valori (low, mid, high), ceea ce sugerează că este derivat dintr-o variabilă numerică, cel mai probabil *skills_count*. Practic, transmite aceeași informație, dar într-o formă simplificată. Dacă le folosim pe ambele în model, riscăm să introducem informație redundantă, ceea ce poate complica inutil modelul fără să îmbunătățească rezultatele.

Pentru a preveni aceste probleme și a îmbunătăți calitatea predicțiilor, se pot aplica următoarele soluții:

- pentru *remote_work*, valorile lipsă pot fi imputate utilizând cea mai frecventă categorie sau prin introducerea unei categorii suplimentare (de exemplu, „Unknown”), astfel încât să nu se piardă informație
- pentru *skill_bracket*, o putem utiliza exclusiv pe ea pentru predicție, fie eliminarea ei în favoarea variabilei numerice *skills_count*, în funcție de performanța obținută în experimente.

3.4 Analiza echilibrului de clase



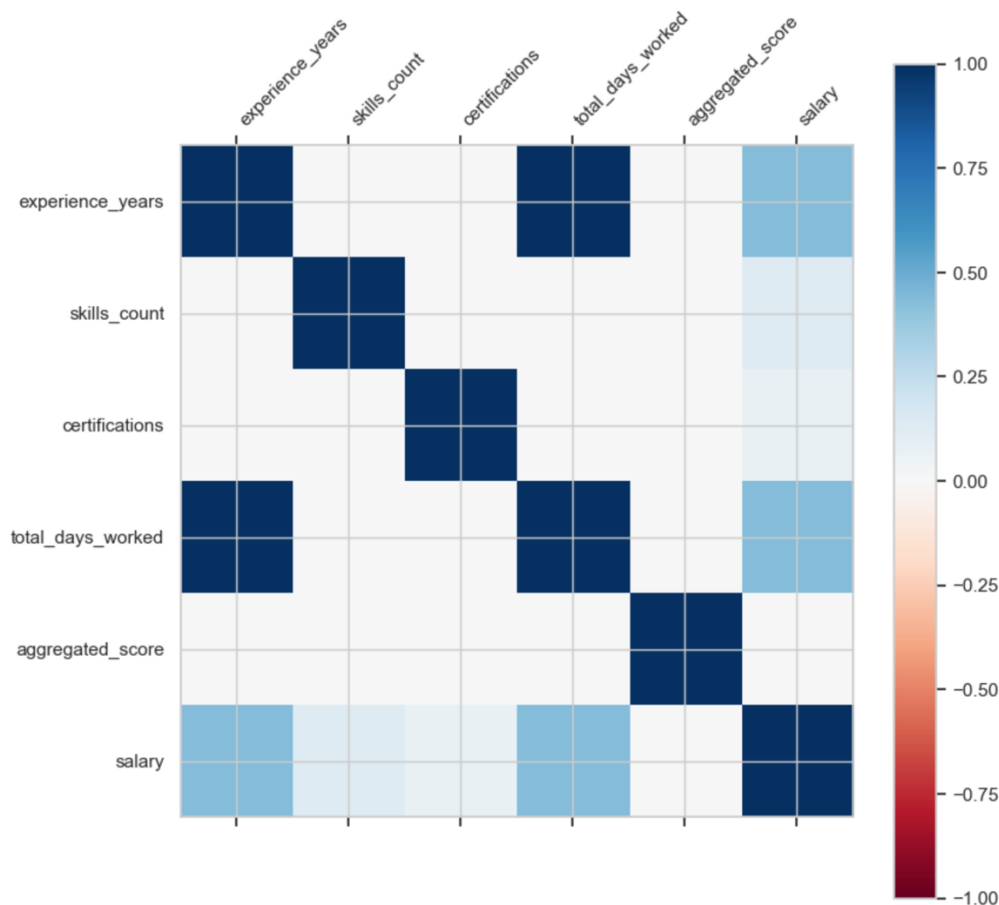
Analiză: Din tabel, cât și din countplot-ul rezultat, tragem concluzia că clasa „No Vacatin” domină cu 34686 (aproximativ 54%) de exemple, comparativ cu celelalte clase ce sunt redade în procent <20%. Asta ne trimite direct la un set de date dezechilibrat.

Așadar, un model cu acest tip de seturi de antrenare care ar prezice mereu „No Vacation” ar obține o acuratețe de 54% fără să fi învățat nimic. În această situație, clasele „Medium” și „Large”, clasele cu cel mai mic suport, sunt cele mai vulnerabile: modelul va tinde să le ignore în favoarea clasei majoritare, producând un recall scăzut.

În concluzie, vom raporta precizie, recall și F1 per clasă, în special pentru „Medium” și „Large”. În plus, vom folosi „class_weight = ,balanced”” în antrenarea modelelor pentru a compensa dezechilibrul.

3.5 Analiza corelației între atribute

- *Analiza de corelație între atributele numerice continue, cât și cu sarcina de regresie.*



	experience_years	skills_count	certifications
experience_years	1.00	0.00	0.00
skills_count	0.00	1.00	0.00
certifications	0.00	0.00	1.00
total_days_worked	1.00	0.00	0.00
aggregated_score	-0.00	0.00	-0.00
salary	0.44	0.13	0.08

	total_days_worked	aggregated_score	salary
experience_years	1.00	-0.00	0.44
skills_count	0.00	0.00	0.13
certifications	0.00	-0.00	0.08
total_days_worked	1.00	-0.00	0.44
aggregated_score	-0.00	1.00	-0.00
salary	0.44	-0.00	1.00

Analiză:

Există o corelație puternică între attributele **experience_years** și **total_days_worked** ($r = 1.00$), sugerând că cele două variabile captează aceeași informație. Acest lucru este explicabil din perspectivă semantică: numărul total de zile lucrate reprezintă, în esență, un calcul al anilor de experiență ($\text{total_days_worked} = \text{experience_years} \times \text{zile lucrate pe an}$).

Includerea simultană a ambelor atribute într-un model de predicție nu aduce o creștere semnificativă a puterii predictive, ci doar amplifică complexitatea. Prin urmare, `total_days_worked` este un candidat pentru eliminare din setul de atribute predictor.

Corelație atribute numerice cu sarcina de regresie (salary)

Examinând corelațiile individuale cu variabila țintă **salary**, se observă că **experience_years** prezintă cel mai ridicat **coeficient de corelație ($r = 0.44$)**, confirmând intuiția că salariul crește odată cu acumularea experienței profesionale.

La polul opus, **aggregated_score** înregistrează o corelație apropiată de zero față de salary, ceea ce indică faptul că acest atribut nu deține putere predictivă relevantă pentru sarcina de regresie. O posibilă explicație constă în faptul că `aggregated_score` este un scor compozit deja standardizat, care reflectă mai degrabă o evaluare de performanță subiectivă.

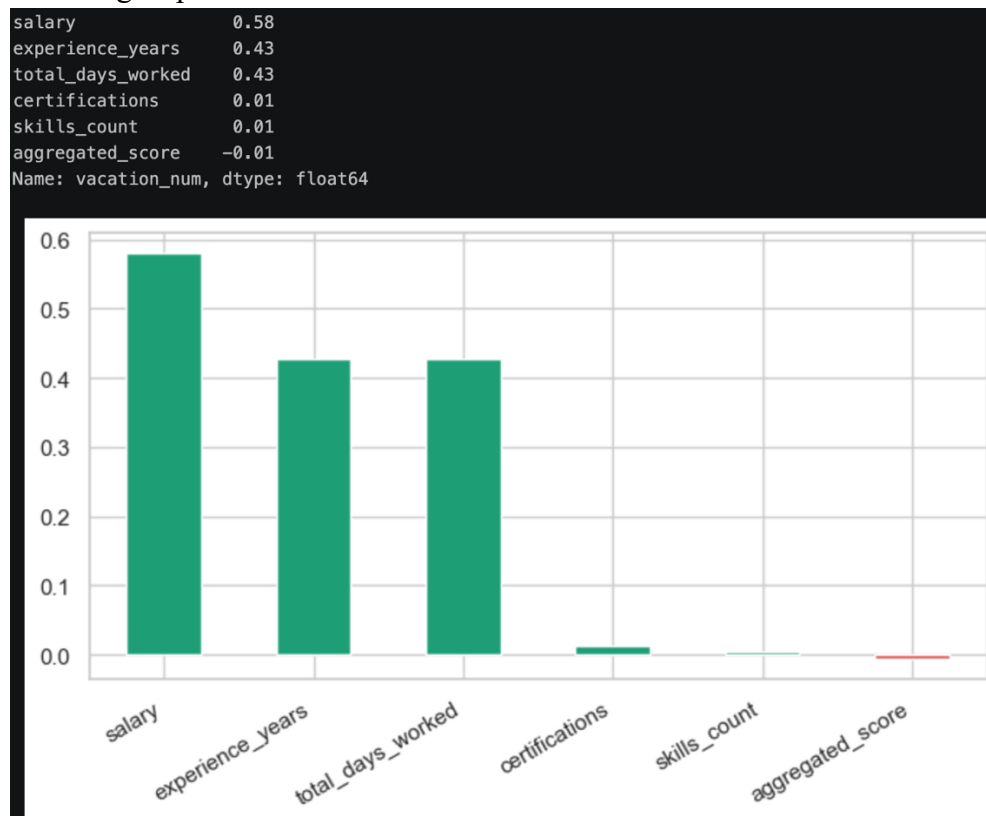
Acesta devine astfel un candidat pentru excludere din modelele de regresie, în scopul simplificării spațiului de atribute fără pierdere de performanță.

- *Analiza de corelație între atributele numerice continue cu sarcina de clasificare.*

În continuare, pentru analiza între atributele numerice și sarcina de clasificare `vacation`, am realizat că corelația Pearson funcționează doar între variabile numerice. ‘vacation’ este o variabilă categorială (‘No Vacation’, ‘Small’, ‘Medium’, ‘Large’), deci nu putem calcula direct corelația Pearson dintre ea și atributele numerice.

Așadar, am transformat ‘vacation’ într-un număr întreg respectând ordinea naturală a claselor (cu cât vacanța este mai mare, cu atât valoarea numerică este mai mare.) Această transformare este justificată tocmai pentru că `vacation` este un atribut ordinal: există o relație clară de ordine `No Vacation < Small < Medium < Large`, reflectând creșterea beneficiilor.

Encoding-ul păstrează această semantică.



Analiză:

Analiza corelației dintre atributele numerice și variabila țintă vacation (encodată ordinal: No Vacation=0, Small=1, Medium=2, Large=3) relevă un pattern clar și interpretabil.

- **salary** arată că este cel mai puternic predictor pentru clasificare, cu $r = 0.58$. Acest rezultat este intuitiv pentru că tipul de vacanță pe care și-o permite un angajat este direct condiționat de puterea sa financiară.
- **experience_years** și **total_days_worked** având coeficienți identici ($r = 0.43$), confirmă redundanța semnalată anterior în matricea de corelație între atribute. Ambele capturează aceeași dimensiune și ambele contribuie în mod egal la predicția tipului de vacanță.
- **certifications** și **skills_count** înregistrează corelații aproape nule ($r = 0.01$), indicând faptul că numărul de certificări sau de competențe tehnice nu influențează semnificativ tipul de vacanță al unui angajat. Aceasta sugerează că beneficiile de tip vacanță sunt determinate mai degrabă de vechime și salariu, nu de profilul tehnic al angajatului.
- **aggregated_score** prezintă o corelație negativă neglijabilă ($r = -0.01$), practic zero, ceea ce confirmă că acest atribut nu aduce valoare predictivă nici pentru clasificare.

În concluzie, pentru sarcina de clasificare, attributele numerice certifications, skills_count și aggregated_score au o contribuție neglijabilă și pot fi eliminate sau păstrate fără impact semnificativ asupra performanței.

○ *Analiza de corelație între attributele categoriale*

Atribut 1	Atribut 2	p-value	Corelație
job_title	education_level	1.00	Nu
job_title	industry	0.08	Nu
job_title	company_size	0.16	Nu
job_title	location	0.49	Nu
job_title	remote_work	0.24	Nu
job_title	skill_bracket	0.42	Nu
education_level	industry	0.96	Nu
education_level	company_size	0.14	Nu
education_level	location	0.60	Nu
education_level	remote_work	0.36	Nu
education_level	skill_bracket	0.10	Nu
industry	company_size	0.66	Nu
industry	location	0.12	Nu
industry	remote_work	0.09	Nu
industry	skill_bracket	0.43	Nu
company_size	location	0.27	Nu
company_size	remote_work	0.31	Nu
company_size	skill_bracket	0.24	Nu
location	remote_work	0.94	Nu
location	skill_bracket	0.52	Nu
remote_work	skill_bracket	0.80	Nu

Analiză:

Nicio pereche de attribute categoriale nu prezintă corelație semnificativă — toate p-value-urile depășesc pragul de 0.05, deci acceptăm H_0 pentru toate combinațiile. Aceasta este o veste favorabilă pentru procesul de modelare: nu există redundanță între attributele categoriale predictor, astfel că toate pot fi păstrate fără a introduce informație duplicată în model. Merită subliniate câteva perechi cu p-value foarte ridicat care confirmă independența clară: education_level și industry ($p = 0.96$), location și remote_work ($p = 0.94$) — deși intuitiv ai fi așteptat ca angajații care lucrează Remote să fie asociați cu o anumită locație, testul arată că în acest dataset cele două attribute sunt distribuite independent.

○ *Analiza de corelație între attributele categoriale și sarcina de clasificare*

Testul Chi-Pătrat a fost aplicat pentru analiza corelației dintre perechi de attribute categoriale, precum și dintre attributele categoriale și variabila țintă de clasificare vacation. Principiul acestui test constă în compararea distribuției observate a valorilor față de distribuția așteptată în ipoteza de independență. Ipoteza nulă H_0 afirmă că cele două variabile sunt independente, adică nu există nicio relație între ele. Dacă $p\text{-value} < 0.05$, respingem H_0 .

și concluzionăm că există o asociere statistică semnificativă între cele două atribute. Cu cât p-value este mai aproape de 0, cu atât corelația este mai puternică.

job_title	p-value = 0.000000	Corelat
education_level	p-value = 0.000000	Corelat
industry	p-value = 0.768810	Independent
company_size	p-value = 0.000000	Corelat
location	p-value = 0.000000	Corelat
remote_work	p-value = 0.000000	Corelat
skill_bracket	p-value = 0.026566	Corelat

Analiză:

job_title, education_level, company_size, location, remote_work și skill_bracket obțin toate p-value = 0.00, respingând cu fermitate ipoteza de independență față de vacanță. Aceasta înseamnă că tipul de vacanță variază semnificativ în funcție de fiecare dintre aceste atribute (toate sunt prin urmare predictorii relevanți pentru sarcina de clasificare). O mențiune aparte merită skill_bracket (p = 0.026), care deși trece pragul de semnificație, are cel mai ridicat p-value dintre atributele corelate (asocierea sa cu vacanță există, dar este mai slabă comparativ cu celelalte).

Singura excepție este industry (p = 0.77), care se dovedește independentă față de vacanță. Industria în care activează angajatul nu influențează semnificativ tipul de vacanță pe care și-o permite — un rezultat contraintuitiv, dar susținut statistic de date.

○ *Analiza de corelație între atributele categoricale și sarcina de regresie*

Testul ANOVA (Analysis of Variance) a fost utilizat pentru analiza relației dintre atributele categoricale și variabila țintă de regresie salary. ANOVA testează dacă media salariului diferă semnificativ între categoriile unui atribut. De exemplu, pentru company_size, testul verifică dacă media salariului la angajații din companii Small diferă semnificativ față de cei din companii Large sau Enterprise. Și aici, H0 afirmă că mediile sunt egale în toate categoriile, deci atributul nu influențează salary. Un p-value < 0.05 respinge această ipoteză și confirmă că atributul este un predictor relevant pentru regresie.

job_title	p-value = 0.000000	Corelat
education_level	p-value = 0.000000	Corelat
industry	p-value = 0.621364	Independent
company_size	p-value = 0.000000	Corelat
location	p-value = 0.000000	Corelat
remote_work	p-value = 0.000000	Corelat
skill_bracket	p-value = 0.000000	Corelat

Analiză:

Rezultatele confirmă un pattern consistent cu cel de la clasificare. **job_title, education_level, company_size, location, remote_work și skill_bracket** sunt toate puternic

corelate cu salary ($p \approx 0.00$), ceea ce este așteptat din perspectivă economică: salariul variază semnificativ în funcție de funcție, nivelul de educație, dimensiunea angajatorului și locație.

Din nou, **industry** se dovedește independentă față de salary ($p = 0.62$), confirmând că industria nu diferențiază semnificativ nivelul de remunerare în acest dataset. Această consistență între cele două sarcini — industry este irelevantă atât pentru clasificare cât și pentru regresie — consolidează și justifică empiric decizia de a elimina industry din ambele modele.

În concluzie, analiza corelațiilor categoriale furnizează două decizii clare de preprocesare: nu există redundanță între attributele categoriale predictor, deci niciun atribut nu trebuie eliminat pe acest criteriu. În schimb, industry este singurul atribut categorial care nu aduce putere predictivă pentru niciuna dintre cele două sarcini și eliminarea sa este justificată empiric prin ambele teste statistice efectuate.

4. Preprocesarea datelor

4.1 Date lipsă pentru un atribut într-un eșantion

Înainte de aplicarea oricărui algoritm de învățare automată, a fost necesară identificarea și tratarea valorilor lipsă din dataset. Analiza inițială a relevat că singurul atribut cu valori lipsă este **remote_work**, cu 19.153 de înregistrări absente, reprezentând aproximativ 30% din totalul exemplurilor de antrenare.

Pentru a decide strategia optimă de imputare, a fost mai întâi analizată natura valorilor lipsă, adică dacă acestea sunt distribuite aleator sau urmează un pattern sistematic legat de alte attribute. Datorită faptului că absența valorii nu este corelată cu alte caracteristici ale angajatului, concluzionăm că nu are sens să folosim celelalte attribute pentru a o estima. Așadar justificăm utilizarea imputării univariante în locul celei multivariante.

Au fost comparate două metode de imputare univariată prin **SimpleImputer**:

Metoda 1 - strategy='most_frequent' (modul): toate cele 19.153 de valori lipsă au fost atribuite categoriei Hybrid, aceasta fiind cea mai frecventă. Rezultatul este însă problematic pentru că Hybrid ajunge la 34.207 exemple, mai mult decât No și Yes combinate, distrugând

complet echilibrul natural al distribuției originale.

```
Valoare imputata (modul): Hybrid
```

```
Distributie dupa imputare:
```

```
remote_work
```

```
Hybrid      34207
```

```
No          14955
```

```
Yes         14838
```

```
Name: count, dtype: int64
```

Metoda 2 - strategy='constant', fill_value='Unknown': valorile lipsă primesc o categorie distinctă Unknown. Distribuția originală rămâne intactă , având Hybrid=15.054, No=14.955, Yes=14.838 ,iar categoria Unknown reflectă faptul că pentru 30% din angajați această informație nu a fost disponibilă la momentul colectării datelor.

```
Distributie dupa imputare:
```

```
remote_work
```

```
Unknown     19153
```

```
Hybrid      15054
```

```
No          14955
```

```
Yes         14838
```

```
Name: count, dtype: int64
```

A fost aleasă imputarea cu *SimpleImputer* cu strategy='constant', fill_value='Unknown' deoarece distribuția originală a atributului remote_work este mai echilibrată între cele trei categorii prezente (aprox. 15.000 exemple fiecare). Imputarea cu modul ar fi atribuit forțat toate cele 19.153 de valori lipsă categoriei Hybrid, dublând frecvența acesteia și dezechilibând distribuția reală. Introducerea categoriei Unknown păstrează integritatea distribuției existente și reflectă mai onest faptul că pentru 30% din angajați această informație nu a fost disponibilă la momentul colectării datelor.

4.2 Valori extreme pentru un atribut într-un eșantion

Identificarea valorilor extreme a fost realizată prin două metode complementare: analiza statisticilor descriptive (*describe()*) și metoda diferenței interquartilice (IQR), cu limitele Q1=0.25 și Q3=0.75.

```
Detectie outlieri IQR standard (Q1=0.25, Q3=0.75)
```

Coloana	Q1	Q3	Lower	Upper	N outlieri
experience_years	5.00	15.00	-10.00	30.00	0
skills_count	5.00	15.00	-10.00	30.00	1909
certifications	1.00	4.00	-3.50	8.50	0
total_days_worked	1200.00	3600.00	-2400.00	7200.00	0
aggregated_score	-0.67	0.68	-2.70	2.71	437
salary	119209.75	169582.25	43651.00	245141.00	587

Analiză:

Din analiza tabelului IQR au rezultat următoarele decizii per atribut:

- **skills_count** : 1.909 outlieri detectați (valori > 30 , limita superioară IQR). Aceștia reprezintă cazuri improbabile de angajați cu un număr excesiv de competențe față de distribuția generală. Îi vom înlocui cu mediana, metodă preferată față de medie deoarece mediana este robustă la valorile extreme . După tratare, $\max=19$, confirmând eliminarea corectă a valorilor extreme.
- **total_days_worked** : metoda IQR nu detectează outlieri (limita inferioară = -2400), însă există 2 valori de -1, imposibile din punct de vedere fizic. Acestea reprezintă anomalii logice, nu statistice, și au fost tratate separat prin înlocuire cu mediana coloanei. După tratare, numărul de valori negative este 0.
- **aggregated_score** : deși prezintă 437 outlieri statistici, coloana a fost eliminată din model în pasul următor (corelație ~ 0 cu ambele target-uri), făcând tratarea outlierilor irelevantă.
- **salary** : prezintă 587 valori peste limita superioară IQR .Deoarece salary este variabila țintă a regresiei, aceste valori nu sunt tratate deoarece reprezintă salarii mari reale și eliminarea sau modificarea lor ar introduce bias în evaluarea modelului.
- **experience_years și certifications** : 0 outlieri, distribuții normale, nu necesită intervenție.

```
Număr outlieri detectați: 1909
Valoare de înlocuire (median): 10.0
Max dupa tratare: 19
Min dupa tratare: 1

Valori negative total_days_worked: 2
Valori negative total_days_worked dupa modificare: 0
```

4.3 Atribute redundante (puternic corelate)

Pe baza analizelor de corelație din secțiunea EDA, au fost eliminate trei atribute din setul de predictori:

- **total_days_worked** : eliminat pe baza corelației foarte ridicate cu **experience_years** ($r = 0.87$), identificată în matricea de corelație. Cele două atribute captează practic aceeași informație: numărul total de zile lucrate este o transformare liniară a anilor de experiență ($\text{total_days_worked} \approx \text{experience_years} \times \text{zile_lucrate_pe_an}$). Includerea ambelor în model nu aduce putere predictivă suplimentară, ci doar complexitate computațională inutilă și risc de multicolaritate.
- **aggregated_score** : eliminat pe baza corelației aproape nule cu ambele target-uri: $r \approx 0.00$ față de salary și $r = -0.01$ față de vacation. Testele au confirmat că acest atribut

nu discriminează între clase și nu contribuie la predicția salariului. Un atribut neinformativ față de obiectiv nu aduce valoare modelului și poate introduce zgomot în antrenare.

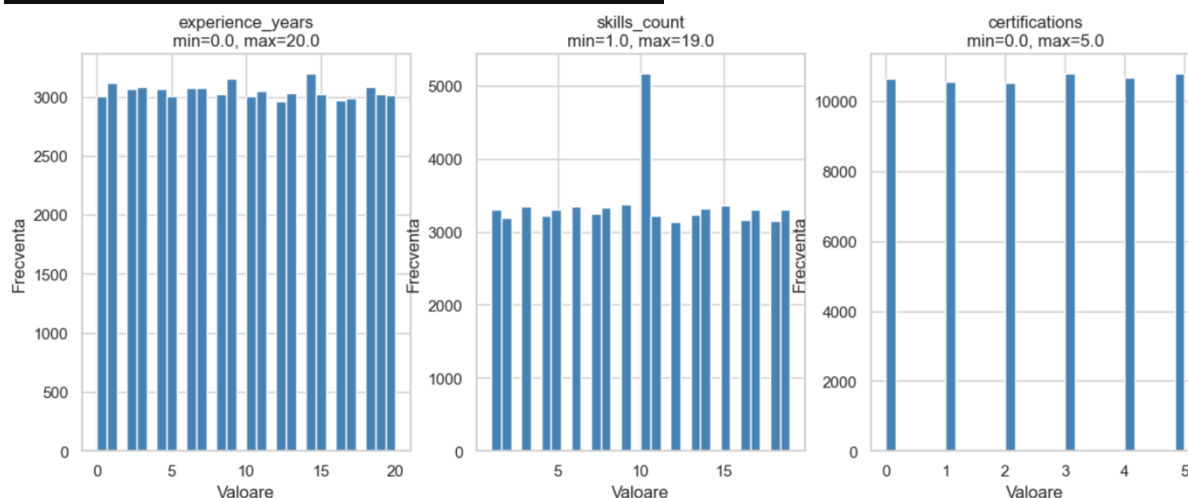
- **Industry** : eliminat pe baza testelor statistice Chi-square față de vacation ($p = 0.77$) și ANOVA față de salary ($p = 0.62$). Ambele teste acceptă ipoteza nulă de independență, confirmând că industria nu influențează semnificativ nici tipul de vacanță, nici nivelul salariului în acest dataset.

Atribute corelate cu target-urile

Conform avertismentului din enunț, nu am eliminat atribute corelate cu obiectivele. **salary** ($r = 0.58$ cu vacation) și **experience_years** ($r = 0.43$ cu vacation, $r = 0.44$ cu salary) sunt puternic corelate cu target-urile, tocmai de aceea sunt predictorii valoroși și au fost păstrați în model.

4.4 Plaje valorile de mărimi diferite pentru valorile numerice

	experience_years	skills_count	certifications
count	64000.00	64000.00	64000.00
mean	9.97	9.98	2.51
std	6.05	5.39	1.71
min	0.00	1.00	0.00
25%	5.00	5.00	1.00
50%	10.00	10.00	3.00
75%	15.00	15.00	4.00
max	20.00	19.00	5.00



Atributele numerice rămase după preprocesare (**experience_years** [0, 20], **skills_count** [1, 19] și **certifications** [0, 5]) se află la ordine de mărime similare, fără diferențe dramatice între ele. Totuși, standardizarea rămâne necesară și justificată din alt motiv: aceste atribute numerice vor fi combinate în model cu atributele encodeate **job_title**, **location** și **remote_work**, care au valori strict mai mici ca 5 în {0, 1, 2, 3, 4}. Față de o valoare de 15 ani de experiență sau 10 skills, o valoare mai mică ca 5 devine neglijabilă într-o combinație liniară, ceea ce ar distorsiona algoritmi sensibili la scală precum Regresia Logistică.

A fost ales `StandardScaler` (transformare la $\text{mean}=0$, $\text{std}=1$) față de alternativele `MinMaxScaler` și `RobustScaler`, deoarece outlierii din `skills_count` au fost deja tratați în pasul anterior, eliminând principalul dezavantaj al `StandardScaler` față de `RobustScaler`. `MinMaxScaler` a fost respins deoarece este sensibil la valorile extreme reziduale, putând comprima artificial distribuția valorilor normale. Un aspect critic respectat în implementare este că `fit()` s-a aplicat exclusiv pe setul de antrenare, iar `transform()` separat pe validare și test, evitând astfel data leakage.

5. Utilizarea algoritmilor de învățare automată

Pregătirea datelor pentru algoritmi de învățare automată

Înainte de a aplica algoritmi de clasificare și regresie, datele necesită o ultimă etapă de transformare. Algoritmi din `scikit-learn` (atât arborii de decizie, cât și regresia liniară, Ridge, Lasso) lucrează exclusiv cu valori numerice, astfel încât toate atributele categorice trebuie convertite într-o formă pe care modelul o poate procesa. Am ales tipul de encoding în funcție de natura semantică a fiecărei coloane.

Encoding ordinal pentru atribute cu ordine naturală. Pentru ***education_level***, ***company_size*** și pentru variabila țintă ***vacation*** am folosit encoding ordinal (un dicționar care mapează fiecare categorie la un întreg). Decizia este justificată de faptul că aceste trei atribute prezintă o relație de ordine implicită care poartă informație utilă pentru model:

- `education_level`: High School (0) < Diploma (1) < Bachelor (2) < Master (3) < PhD (4)
- `company_size`: Startup (0) < Small (1) < Medium (2) < Large (3) < Enterprise (4)
- `vacation`: No Vacation (0) < Small (1) < Medium (2) < Large (3)

Pentru `job_title`, `location` și `remote_work`, categoriile nu au o ordine naturală („Data Scientist > Backend Developer” sau că „USA > Germany” nu sunt adevărate). Atribuirea unor numere arbitrare ar introduce relații numerice false pe care modelul le-ar interpreta ca informație reală. De aceea am aplicat one-hot encoding (`pandas.get_dummies`), care creează câte o coloană binară pentru fiecare categorie, fiecare exemplu având valoarea 1 doar pentru categoria sa și 0 pentru restul. Am folosit parametrul `drop_first=True` pentru a elimina prima coloană din fiecare grup, prevenind astfel dummy variable trap (multicolinearitate perfectă) astfel că știind valorile a $n-1$ coloane, a n -a se deduce automat, iar păstrarea ei ar introduce redundanță inutilă.

Am adăugat un assert la finalul etapei pentru a verifica explicit că nu au rămas valori NaN în setul de train.

Separarea features–target : Înainte de encoding, am extras variabilele țintă (`y_clf` pentru clasificare, `y_reg` pentru regresie) și le-am mapat la valori numerice. Apoi am eliminat din features atât țintele, cât și `skill_bracket` (redundant — acest atribut este o discretizare

directă a lui `skills_count`, deci păstrarea ambelor ar introduce multicolinearitate fără câștig informațional). Eliminarea țințelor din features este o precauție elementară: dacă modelul ar avea acces la salariu când prezice tipul de vacanță (sau invers), ar atinge performanță aproape perfectă pe baza unei „scurtături” care nu există în setul de testare real — o formă clasică de data leakage.

Shape train: (51200, 27), val: (12800, 27), test: (16000, 27) Total features: 27			13	job_title_Product Manager
			14	job_title_Software Engineer
			15	location_Canada
			16	location_Germany
			17	location_India
			18	location_Netherlands
			19	location_Remote
			20	location_Singapore
			21	location_Sweden
			22	location_UK
			23	location_USA
			24	remote_work_No
			25	remote_work_Unknown
			26	remote_work_Yes

Standardizare numerică: După encoding, attributele numerice rămase au scale foarte diferite: `experience_years` se exprimă în zeci, `skills_count` în zeci, iar coloanele rezultate din one-hot sunt binare (0/1). De aceea am aplicat `StandardScaler`, subiect discutat la capitolul anterior, care transformă fiecare coloană la medie 0 și deviație standard 1, aducând astfel toate attributele pe o scală comparabilă.

Anti-leakage: scaler fitat exclusiv pe train. Scaler-ul a fost fitat doar pe `X_train` (pentru a calcula media și deviația standard) și apoi aplicat (transform) identic pe `X_val` și `X_test`. Aceeași logică s-a aplicat anterior și la imputarea valorilor lipsă, respectiv la detecția de outliers, toate transformările sunt învățate din train și aplicate pe restul.

```
X_train = df_train.values.astype(float)
X_val   = df_val.values.astype(float)
X_test  = df_test.values.astype(float)

print(f'Train: {X_train.shape[0]} ex.  Val: {X_val.shape[0]} ex.  Test: {X_test.shape[0]} ex.')

# Standardizare – scaler fitat doar pe train, aplicat pe toate.
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_val_scaled   = scaler.transform(X_val)
X_test_scaled  = scaler.transform(X_test)
```

✓ 0.2s

Train: 51200 ex. Val: 12800 ex. Test: 16000 ex.

În continuare, voi prezenta modul de gândire și parcursul experimental prin care am ajuns la modelele finale, atât pentru clasificare, cât și pentru regresie. Abordarea respectă

metodologia ablației descrisă în enunț: pornesc de la un baseline simplu, variez câte un singur hiperparametru pe rând, jurnalizez rezultatele și păstrez acea modificare doar dacă aduce un câștig real.

Pentru evaluarea modelelor folosesc setul de validare obținut din 20% date al setului de antrenare (train.csv).

5.1. Clasificare

Sarcina este de clasificare multclasă cu 4 clase (No Vacation, Small, Medium, Large), pe un set dezechilibrat unde clasa majoritară No Vacation reprezintă aprox 54% din exemple. Prin urmare, accuracy singură nu este suficientă, deoarece un clasificator naiv care prezice mereu clasa majoritară ar obține deja aproximativ 54%. Am ales să raportez în plus F1 macro (medie neponderată per clasă) și F1 individual pe clasele minoritare, pentru a măsura corect și performanța pe Medium și Large.

5.1.1. Baseline cu Decision Tree

Conform cerinței minime din enunț, am pornit cu un DecisionTreeClassifier configurat cu hiperparametri implicați. Rezultatul pe validare a fost o acuratețe de aprox. 67% și un F1 macro de aprox. 0.57. Modelul depășește deja pragul de 0.6 menționat în baremul de perfecționare, dar are limite clare: arborele crește fără constrângeri de adâncime, ceea ce duce la overfitting (acuratețea pe train este aproape de 100%, mult peste cea pe validare).

Accuracy: 0.6727				
F1 macro: 0.5730				
	precision	recall	f1-score	support
No Vacation	0.85	0.84	0.84	6937
Small	0.43	0.44	0.44	2575
Medium	0.37	0.37	0.37	1706
Large	0.64	0.64	0.64	1582
accuracy			0.67	12800
macro avg	0.57	0.57	0.57	12800
weighted avg	0.68	0.67	0.67	12800

5.1.2 Variația max_depth

Următorul hiperparametru pe care l-am studiat este adâncimea maximă a arborelui. Am rulat experimente pentru valorile {2, 5, 10, 20, 25}. Concluzia: la adâncimi mici (2-5) modelul face underfitting, F1 macro rămâne sub 0.5; la adâncimi mari (20-25) modelul memorează datele de train și performanța pe validare scade. Optimul a fost la max_depth=10,

unde am obținut un echilibru bun între bias și varianță.

	Accuracy (↑)	F1 macro (↑)	F1 No Vacation (↑)	F1 Large (↑)
max_depth				
2	0.610300	0.331700	0.768300	0.558500
5	0.712300	0.593900	0.880800	0.684500
10	0.742300	0.646100	0.890700	0.719100
20	0.681900	0.585900	0.847100	0.663300
25	0.671300	0.571300	0.843000	0.642200

5.1.3. Variația min_samples_leaf

Am testat valorile {2, 5, 10, 15, 20, 30}. Rezultatul a fost interesant: diferențele între aceste valori sunt nesemnificative (acuratețea variază cu mai puțin de 0.5%). Concluzia mea este că odată ce max_depth=10 este fixat, complexitatea arborelui este deja suficient de constrânsă încât numărul minim de exemple pe frunză nu mai are impact major. Am ales min_samples_leaf=30 pentru a fi pe partea conservatoare (frunze mai stabile statistic).

	Accuracy (↑)	F1 macro (↑)	F1 Large (↑)
min_samples_leaf			
2	0.742000	0.645400	0.718600
5	0.743600	0.646600	0.721100
10	0.744400	0.646500	0.718600
15	0.744500	0.648500	0.726200
20	0.746000	0.649800	0.722900
30	0.747600	0.653100	0.725700

5.1.4. Variația class_weight

Având în vedere dezechilibrul claselor, am testat class_weight='balanced' versus None. După observarea tabelului, observăm că opțiunea 'balanced' scade acuratețea generală (de la aprox 0.74 la aprox 0.72) dar crește F1 pe clasele minoritare. Asta se întâmplă pentru că modelul ponderat este forțat să fie mai atent la clasele cu suport mic, în detrimentul clasei majoritare. Am decis să păstrez class_weight='balanced' deoarece prioritatea este performanța echilibrată pe toate clasele, nu maximizarea accuracy-ului.

	Accuracy (↑)	F1 macro (↑)	F1 No Vacation (↑)	F1 Large (↑)
class_weight				
None	0.747600	0.653100	0.894700	0.725700
balanced	0.723400	0.658800	0.866100	0.725100

5.1.5. Random Forest

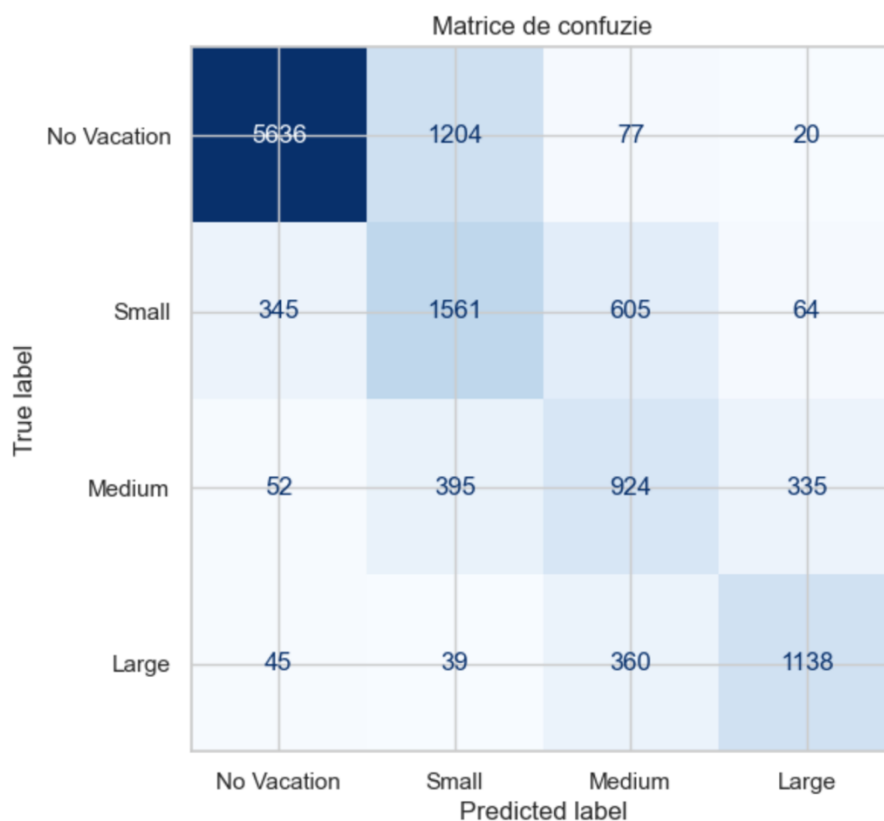
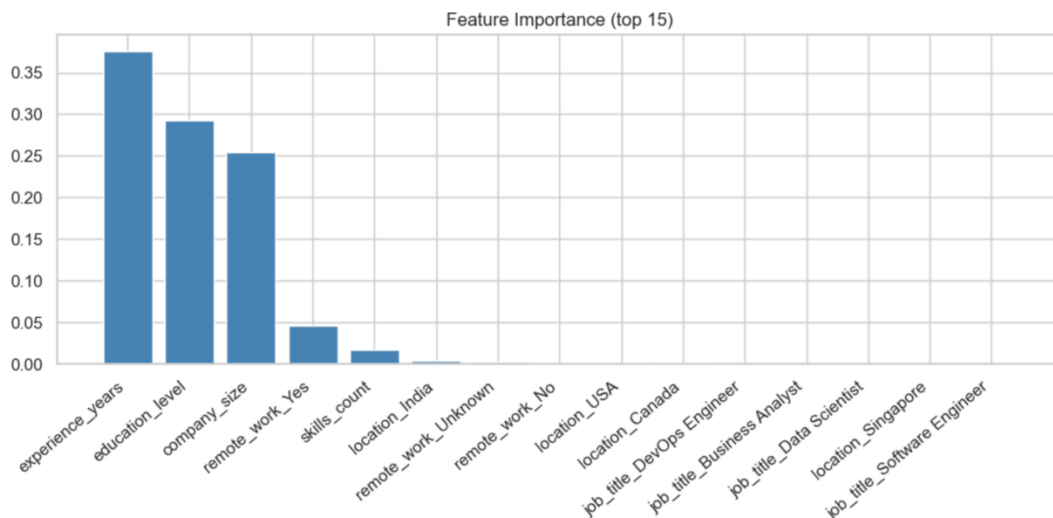
După ce am stabilit cei trei hiperparametri optimi pentru un Decision Tree singur, am trecut la un ansamblu de arbori (RandomForestClassifier, n_estimators=200) cu aceeași configurație. Random Forest reduce varianța prin medierea predicțiilor mai multor arbori antrenați pe subseturi diferite ale datelor, ceea ce duce la o îmbunătățire clară: acuratețea urcă la aprox 73% iar F1 macro la 0.65.

	Accuracy (↑)	F1 macro (↑)
Configurație		
1. Baseline DecisionTree	0.671200	0.572600
2. + max_depth=10	0.742300	0.646000
3. + min_samples_leaf=30	0.747600	0.653100
4. + class_weight='balanced'	0.723400	0.658800
5. RandomForest	0.724200	0.648400

Concluzii pentru clasificare:

Modelul final ales este **Decision Tree cu max_depth=10, min_samples_leaf=30, class_weight='balanced'**. Tabelul cumulativ de ablație arată că, deși configurația fără class_weight obține o acuratețe ușor mai mare (0.7505 vs 0.7323), F1 macro este mai relevant într-un set dezechilibrat: o acuratețe ridicată poate fi obținută trivial prezicând doar clasa majoritară. Configurația cu class_weight='balanced' îmbunătățește F1 macro, ceea ce înseamnă predicții mai echilibrate pe clasele minoritare Small, Medium și Large. Trecerea la Random Forest nu a adus câștig în experimentele mele și a fost respinsă pe baza datelor. Analizele făcute după modelul ales sunt redată de:

	precision	recall	f1-score	support	Performanță pe setul de test			
					precision	recall	f1-score	support
No Vacation	0.93	0.81	0.87	6937	No Vacation	0.92	0.81	0.86
Small	0.49	0.61	0.54	2575	Small	0.48	0.61	0.54
Medium	0.47	0.54	0.50	1706	Medium	0.49	0.56	0.52
Large	0.73	0.72	0.73	1582	Large	0.77	0.74	0.75
accuracy			0.72	12800	accuracy			0.73
macro avg	0.65	0.67	0.66	12800	macro avg	0.67	0.68	0.67
weighted avg	0.75	0.72	0.73	12800	weighted avg	0.76	0.73	0.74



5.2. Regresie

Sarcina este predicția unei variabile numerice continue (salary) pe baza atributelor angajatului. Aici metricile relevante sunt MAE (eroarea medie absolută, în unități de salariu), RMSE (penalizează mai mult erorile mari) și R^2 (proporția din varianța datelor explicată de model).

5.2.1. Baseline cu Linear Regression

Am pornit cu o regresie liniară simplă fără regularizare. Rezultatul a fost surprinzător de bun: $R^2 = 0.948$ pe validare, $RMSE = 8470$, mult sub pragul de perfecționare de 10000 din enunț. Diferența între eroarea pe train ($RMSE = 8527$) și cea pe validare este mică, ceea ce ne indică faptul că modelul nu face overfitting, așadar relația dintre features și salary este într-adevăr aproape liniară în acest dataset.

```
Baseline: LinearRegression
Train          MAE=6609  RMSE=8527  R^2=0.9479
Validation     MAE=6592  RMSE=8470  R^2=0.9489
```

5.2.2. Regularizare Ridge (L2) și Lasso (L1)

Conform cerinței din enunț, am variat factorul de regularizare $\alpha \in \{0.01, 0.1, 1, 10, 100, 1000\}$ pentru ambele tipuri de regularizare. Gândirea mea inițială a fost certificată de faptul că dacă regresia liniară simplă deja generalizează aproape perfect, regularizarea ar trebui să aducă îmbunătățiri marginale sau chiar nule. Așadar, am încercat să demonstrez experimental:

- Pentru α mic (0.01-10), Ridge și Lasso produc rezultate identice cu LinearRegression ($RMSE = 8470$, $R^2 = 0.948$).
- Pentru α mare (100 -1000), performanța scade semnificativ (Lasso ajunge la $RMSE$ 11345, R^2 aprox 0.91), deoarece regularizarea agresivă forțează prea multe coeficiente la zero.

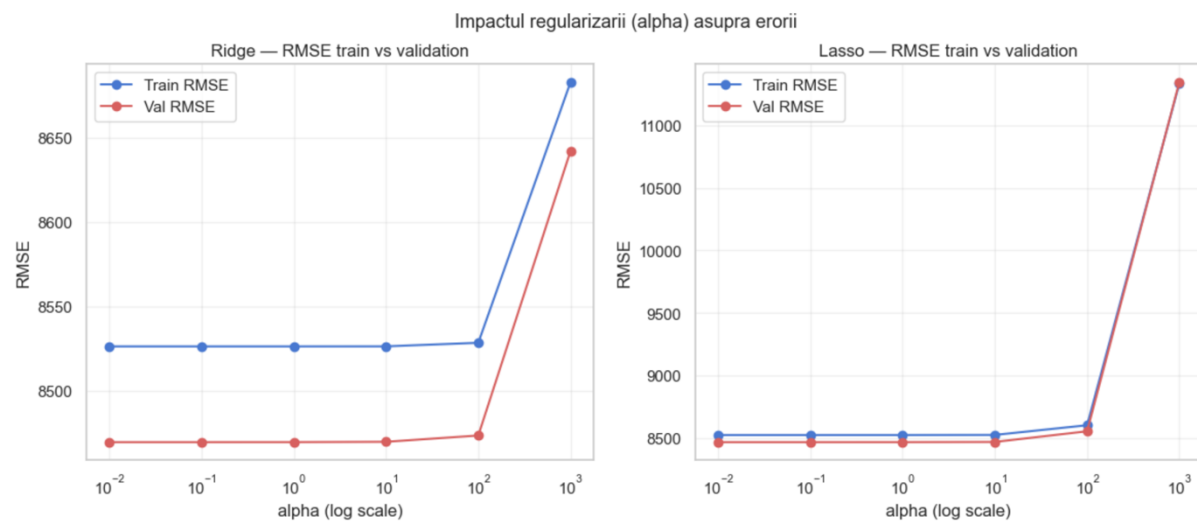
Ridge – RMSE train vs val per alpha				Lasso – RMSE train vs val per alpha			
	train_rmse	val_rmse	val_r2		train_rmse	val_rmse	val_r2
alpha				alpha			
0.01	8527.0	8470.0	0.9489	0.01	8527.0	8470.0	0.9489
0.10	8527.0	8470.0	0.9489	0.10	8527.0	8470.0	0.9489
1.00	8527.0	8470.0	0.9489	1.00	8527.0	8470.0	0.9489
10.00	8527.0	8470.0	0.9489	10.00	8527.0	8471.0	0.9489
100.00	8529.0	8474.0	0.9488	100.00	8605.0	8557.0	0.9478
1000.00	8683.0	8642.0	0.9468	1000.00	11334.0	11345.0	0.9083

```
Cel mai bun alpha Ridge: 0.01
Cel mai bun alpha Lasso: 0.01
```

Concluzia mea despre regularizare:

Pe acest dataset, regularizarea nu aduce câștig deoarece numărul de features (27 după one-hot encoding) este mic în raport cu numărul de exemple de antrenare (51200), iar features-urile nu sunt nici puternic corelate între ele. Regularizarea ar avea valoare adăugată într-un dataset cu sute sau mii de features, sau cu multicolinearitate puternică, dar nu este

cazul aici.



Curbele de eroare train vs val (graficele cu RMSE în funcție de alpha pe scară logaritmică) arată clar acest pattern: cele două curbe rămân lipite pentru toate valorile mici ale lui alpha și diverg doar la valori extreme, semnătură vizuală a unui model care nu suferă de overfitting.

Concluzii pentru regresie:

	MAE val	RMSE val	R ² val	RMSE test	R ² test
Model					
LinearRegression	6592.000000	8470.000000	0.948900	8432.000000	0.948500
Ridge(alpha=0.01)	6592.000000	8470.000000	0.948900	8432.000000	0.948500
Lasso(alpha=0.01)	6592.000000	8470.000000	0.948900	8432.000000	0.948500

Cel mai bun model: LinearRegression

Modelul final ales este **Linear Regression simplă** (echivalent cu Ridge/Lasso la alpha mic).

Performanța finală: RMSE = 8432, MAE = 6592, R² = 0.9489 pe validare și R² = 0.9485 pe test. Modelul depășește pragul de perfecționare (RMSE < 10000) și generalizează identic pe validare și pe test.

R² final pe validare: 0.9489
R² final pe test: 0.9485

Principalele motive pentru care avem o performanță atât de bună este că:

- preprocesarea atentă a eliminat aggregated_score și total_days_worked (perfect corelat cu experience_years);
- standardizarea valorilor numerice a permis regresiei liniare să cântărească corect contribuțiile fiecărui feature;
- relația dintre experiența profesională, nivelul educațional, locație și salariu este intrinsec liniară în acest dataset ,deci un model simplu este suficient.

O lecție importantă pe care am concluzionat-o după această analiză este dată de faptul că modelul mai simplu nu este modelul mai prost. Dacă datele permit, o regresie liniară este preferabilă unui dataset complex, pentru că este mai rapidă, mai interpretabilă, și nu introduce varianță suplimentară.

